



Università degli Studi di Torino
Corso di Laurea Triennale in Informatica

**Studio e sperimentazione degli
approcci di salvataggio dello stato di gioco**

Tesi di Laurea

Relatore/Relatrice
Cavagnino Davide

Candidato/a
Novello Gabriele
976994

Anno Accademico 2023/2024

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

Abstract

Questa tesi esplora la progettazione e l'implementazione di un sistema di salvataggio scalabile e retrocompatibile per il videogioco *Hoarde*, sviluppato con il game engine Godot. L'obiettivo principale è stato creare un sistema di salvataggio flessibile e sicuro che permetta ai giocatori di salvare e caricare i loro progressi e che sia in grado di adattarsi a modifiche e aggiornamenti futuri del gioco, mantenendo al contempo la compatibilità con le versioni precedenti. Sebbene il sistema sviluppato presenti caratteristiche specifiche per il videogioco in questione e per il game engine utilizzato, i problemi affrontati e le soluzioni proposte possono essere applicati anche in contesti più generali.

Dopo una descrizione del game engine e del videogioco, si approfondiscono i vari aspetti della progettazione di un sistema di salvataggio, come la scelta del formato del file e le considerazioni sulla sicurezza. Si procede poi con una spiegazione di alcuni concetti interni al game engine utilizzati per l'implementazione del sistema di salvataggio, descritta in dettaglio in seguito. Al fine di garantire la retrocompatibilità, vengono inoltre presentate soluzioni a problematiche tipiche che possono sorgere in caso di modifiche o aggiornamenti del gioco.

In conclusione, vengono delineati possibili sviluppi futuri al fine di migliorare ulteriormente il sistema di salvataggio.

Indice

1	Introduzione	3
2	Descrizione degli strumenti usati	5
2.1	Godot	5
2.1.1	Nodi e scene	6
2.1.2	SceneTree	8
2.2	Asset	8
3	Applicazione creata	9
3.1	Meccaniche di gioco	10
3.2	Nemici	11
4	Sistema di salvataggio	13
4.1	Formati di file	13
4.1.1	Formato JSON	14
4.1.2	File binari	14
4.1.3	Altri formati	15
4.1.4	Confronto tra JSON e binario	15
4.1.5	Conclusione	16
4.2	Sicurezza	16
4.3	Struttura	17
4.4	Salvataggio	18
4.4.1	SaveManager e autoload	18
4.4.2	Struttura del file	19
4.4.3	Gruppi	20
4.4.4	Implementazione	21
4.5	Caricamento	24
4.5.1	Implementazione	25
4.6	Retrocompatibilità	29
4.6.1	Aggiunta di proprietà	29
4.6.2	Rimozione di proprietà	31
4.6.3	Rinomina di proprietà	32
4.6.4	Cambio di dominio di proprietà	33
5	Conclusione	35

Capitolo 1

Introduzione

Negli ultimi anni, il mercato videoludico ha conosciuto una crescita esponenziale, alimentata non solo dalla costante evoluzione tecnologica, ma anche dall'espansione delle piattaforme di distribuzione digitale, dall'aumento dell'accessibilità agli strumenti di sviluppo e dall'incremento dell'interesse del pubblico per esperienze di gioco sempre più variegate. Grazie a un'ampia gamma di titoli, che spaziano dai giochi mobile di piccola scala a progetti fotorealistici ad alto budget, l'industria è riuscita a coinvolgere un pubblico sempre più vasto. Ogni tipologia di videogioco presenta caratteristiche distintive e il suo sviluppo richiede il coinvolgimento di diverse discipline tecniche e artistiche, come game design, programmazione, illustrazione, scrittura, audio, musica e molte altre aree. Questa complessità rende il processo di creazione estremamente articolato, con ciascun aspetto che gioca un ruolo fondamentale nel garantire un'esperienza di gioco coerente e coinvolgente.

Tra le numerose componenti che caratterizzano un videogioco, emergono elementi chiave come la grafica, il gameplay e l'interazione con il giocatore. Tuttavia, il tema trattato in questa tesi riguarda un elemento spesso dato per scontato, il sistema di salvataggio. Quest'ultimo riveste un'importanza cruciale ed è presente in quasi tutti i titoli. Questo sistema non solo consente ai giocatori di mantenere i progressi ottenuti, ma assicura anche una continuità nell'esperienza di gioco. La sua funzione diventa particolarmente vitale nei titoli soggetti a frequenti aggiornamenti o modifiche, dove una gestione efficace dei salvataggi è essenziale per preservare l'integrità del gioco e garantire la soddisfazione del giocatore.

L'obiettivo di questa tesi è quindi la progettazione e l'implementazione di un sistema di salvataggio per il videogioco *Hoarde*, sviluppato utilizzando il game engine Godot. Le caratteristiche fondamentali di questo sistema includono:

- **Scalabilità:** il sistema deve poter facilmente adattarsi all'aggiunta di nuove funzionalità del gioco o la modifica di quelle esistenti.
- **Sicurezza:** deve garantire la protezione dei dati, prevenendo perdite accidentali e assicurando l'integrità dei file di salvataggio.

- **Retrocompatibilità:** il sistema deve assicurare la compatibilità dei vecchi file di salvataggio anche dopo aggiornamenti significativi del gioco, evitando la perdita di progressi da parte dei giocatori.

Di seguito viene fornita una breve descrizione dei contenuti di questo documento:

Capitolo 2. Descrizione degli strumenti usati. Analizza gli strumenti utilizzati per lo sviluppo del gioco, tra cui il game engine Godot e i vari asset grafici. Si pone particolare attenzione alle principali componenti dell'engine, che costituiranno la base per funzionalità più complesse introdotte nei capitoli successivi.

Capitolo 3. Applicazione creata. Fornisce una panoramica del videogioco *Hoarde*, evidenziando le sue caratteristiche principali.

Capitolo 4. Sistema di salvataggio. Approfondisce le scelte progettuali e implementative del sistema di salvataggio, fornendo una visione dettagliata delle tecniche utilizzate.

Capitolo 5. Conclusione. Riassume i risultati ottenuti e offre riflessioni sul futuro sviluppo del sistema.

Capitolo 2

Descrizione degli strumenti usati

In questo capitolo vengono introdotti tutti gli strumenti utilizzati per lo sviluppo dell'applicazione trattata in questa tesi. La prima sezione introduce il principale strumento utilizzato, Godot, e le sue principali caratteristiche, che saranno alla base dei concetti più complessi spiegati nei capitoli a seguire. La seconda sezione, descriverà le varie fonti di provenienza dei singoli componenti grafici del gioco.

2.1 Godot

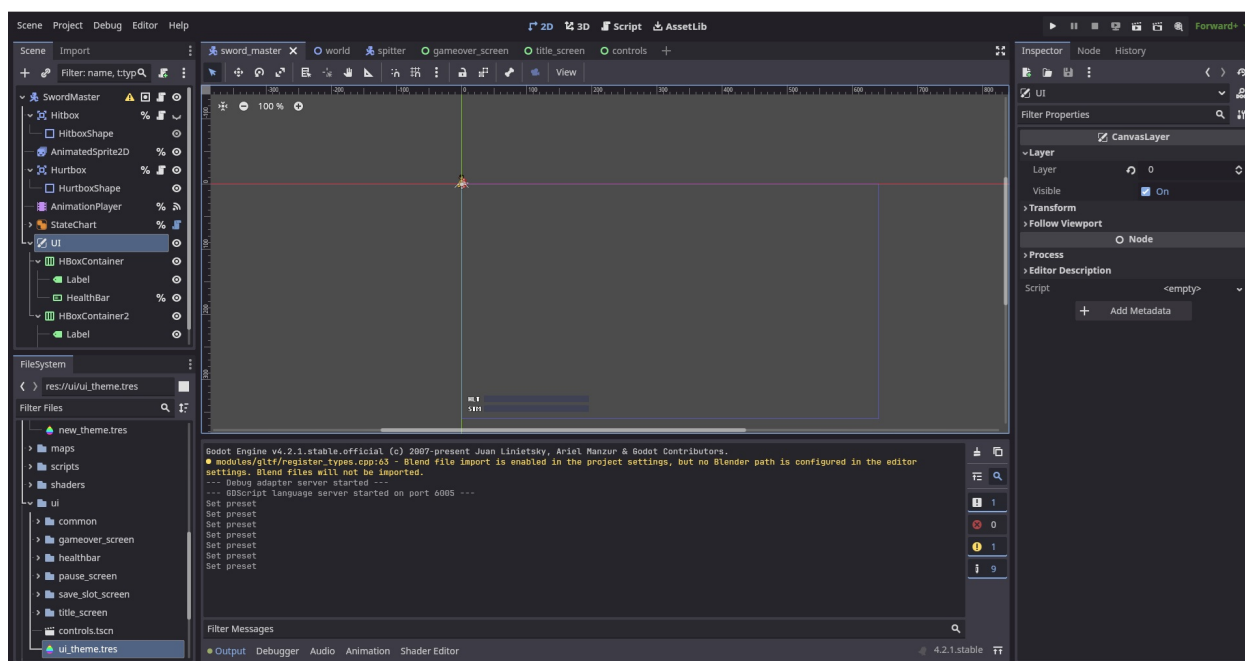


Figura 2.1.1: Interfaccia di Godot

Godot [1] è un game engine open source che ha visto una rapida crescita in popolarità negli ultimi anni. Progettato per lo sviluppo di videogiochi 2D e 3D, Godot fornisce un ambiente integrato

(IDE) con strumenti avanzati per gestire ogni aspetto del processo creativo.

Una delle sue caratteristiche distintive è l'architettura a nodi, che organizza i componenti del gioco in una struttura gerarchica modulare. Questo approccio semplifica la progettazione, la manutenzione e l'espansione dei progetti, ed è una delle principali caratteristiche che lo differenzia dagli altri game engine esistenti. Godot supporta diversi linguaggi di programmazione per gestire i comportamenti e le interazioni dei nodi, tra cui C# e GDScript, un linguaggio di scripting proprietario, integrato direttamente nell'engine, simile a Python. Nonostante C# offra prestazioni superiori, per lo sviluppo di questa applicazione è stato utilizzato GDScript grazie alla sua rapidità e facilità di utilizzo, che meglio risponde alle esigenze del progetto.

Uno screenshot di come si presenta Godot è mostrato in Figura 2.1.1.

È importante sottolineare che Godot rappresenta solo uno degli strumenti disponibili per il raggiungimento degli scopi di questa ricerca; infatti, le stesse tecniche e approcci discussi potrebbero essere applicati anche ad altri game engine esistenti.

2.1.1 Nodi e scene

Nel corso di questo documento verranno spesso introdotte alcune delle funzionalità e strutture fondamentali dell'engine ritenute necessarie per comprendere appieno le scelte di sviluppo effettuate. Tra queste, i nodi e le scene costituiscono la base, ed è quindi essenziale presentarli fin da subito. Sebbene non entreremo nei dettagli più tecnici e complessi, queste nozioni sono indispensabili per comprendere gli aspetti tecnici alla base del sistema di salvataggio. Per eventuali approfondimenti, verranno forniti rimandi alla documentazione ufficiale di Godot [2], che offre descrizioni più approfondite.

I nodi sono gli elementi fondamentali alla base di qualsiasi gioco sviluppato in Godot. Ogni nodo rappresenta una singola unità funzionale del gioco e ne esistono decine di tipologie diverse, ciascuna progettata per svolgere compiti specifici, come visualizzare immagini, riprodurre suoni, rappresentare una telecamera virtuale e molti altri. Ogni nodo, di qualsiasi tipologia, presenta le seguenti caratteristiche:

- possiede una serie di proprietà modificabili
- prevede la possibilità di aggiungere uno script per programmare il suo comportamento
- può essere esteso con nuove proprietà e funzioni
- può essere aggiunto come figlio di altri nodi

Quest'ultima proprietà permette infatti di organizzare i nodi in strutture ad albero, rendendo possibile creare combinazioni complesse. L'esempio in Figura 2.1.2 mostra un esempio di combinazione di nodi ai fini di rappresentare un personaggio controllabile dal giocatore.

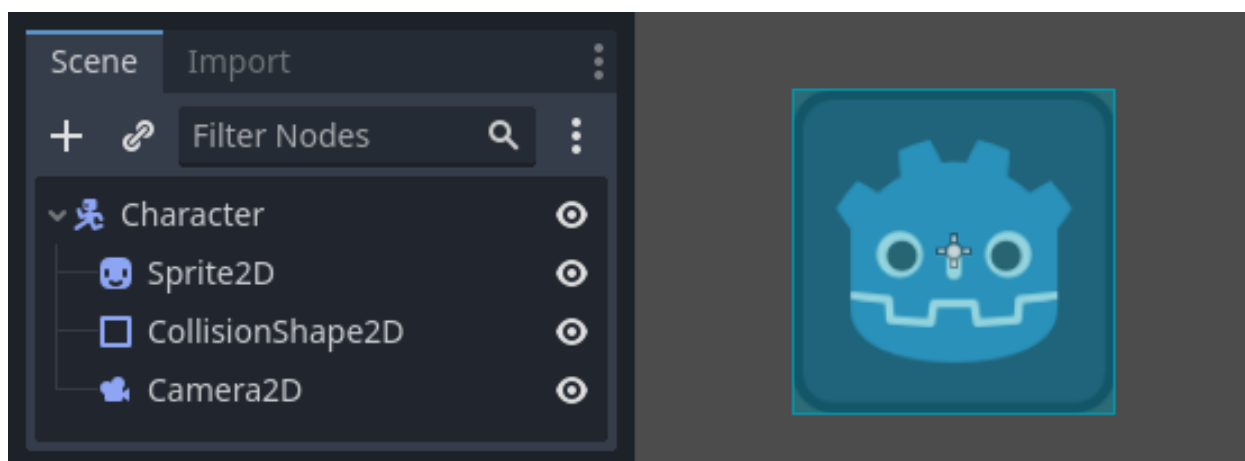


Figura 2.1.2: Combinazioni di diverse tipologie di nodi permettono di creare nodi più complessi. In questo esempio, un nodo `CharacterBody2D`, specializzato per gestire il movimento di un personaggio controllato dal giocatore, viene espanso, aggiungendo una sua rappresentazione visiva con un'immagine o *sprite* (`Sprite2D`), un *collider* per interagire fisicamente con altri oggetti (`CollisionShape2D`) e una telecamera virtuale (`Camera2D`) che seguirà il personaggio.

Inoltre, al fine di poter riutilizzare e istanziare più volte queste combinazioni di nodi all'interno dello stesso progetto, Godot permette di salvarle sottoforma di file chiamati scene (Figura 2.1.3).

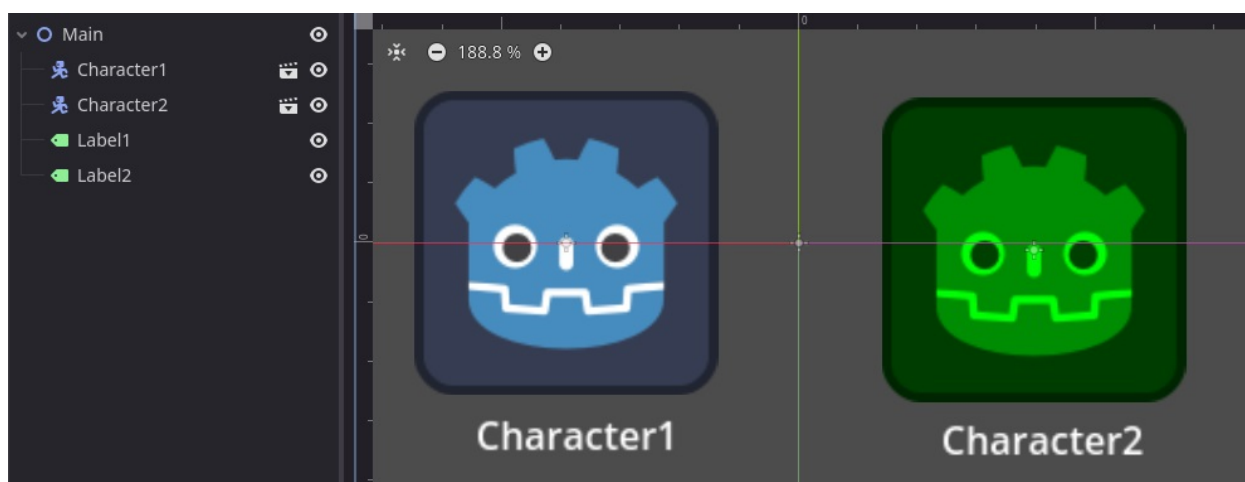


Figura 2.1.3: È possibile salvare la combinazione di nodi mostrata precedentemente in Figura 2.1.2 in una scena `character.tscn` e istanziarla più volte, permettendo la creazione di più personaggi. I due personaggi sono istanze completamente indipendenti l'una dall'altra che utilizzano la stessa struttura di nodi.

Questo principio si applica a qualsiasi tipo di struttura in Godot, dai personaggi, agli elementi dell'interfaccia utente, fino ai componenti del mondo di gioco.

2.1.2 SceneTree

Tutti i nodi e le scene, se inseriti all'interno della scena principale, chiamata **SceneTree**, diventano attivi. Questa struttura ad albero si occuperà dell'esecuzione del *main loop* dell'applicazione, gestendo il rendering e l'aggiornamento di tutti i nodi attivi, l'elaborazione degli input e molto altro. Il **SceneTree** è gestito come un vero e proprio albero, consentendo di aggiungere e rimuovere nodi e scene a tempo di esecuzione, garantendo una gestione efficiente degli elementi e delle risorse del gioco [3][4].

2.2 Asset

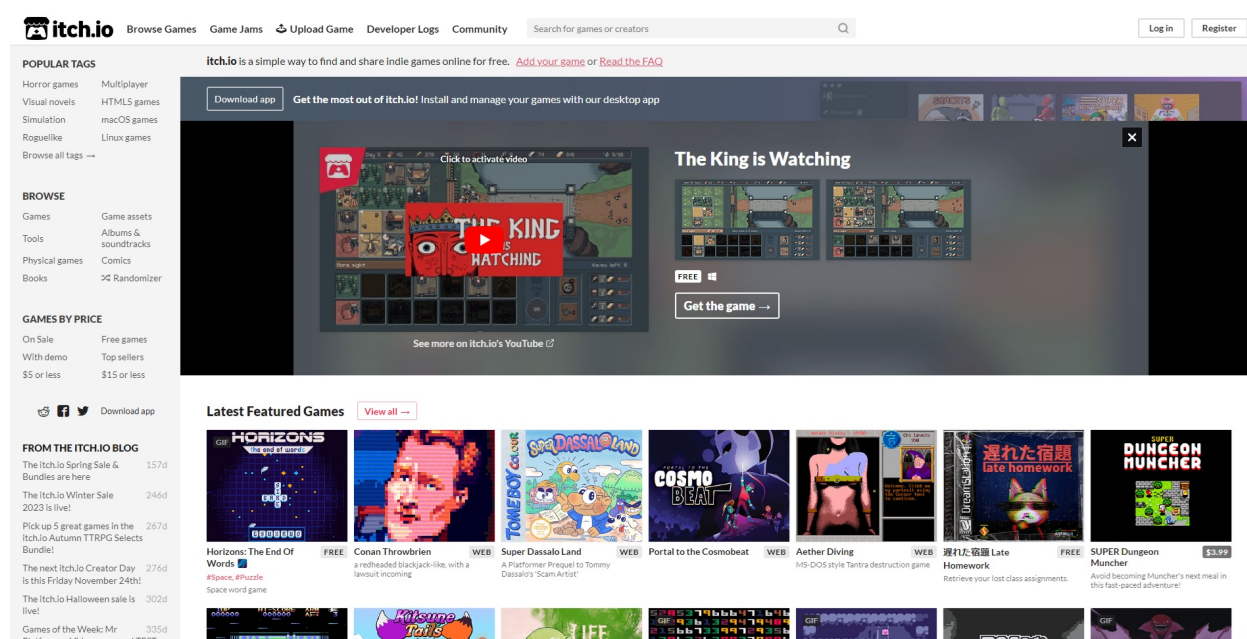


Figura 2.2.4: Homepage di itch.io

Tutti gli *asset*¹ utilizzati per la mappa, i personaggi e le animazioni all'interno dell'applicazione sono stati ottenuti tramite la piattaforma itch.io [5]. Itch.io è un marketplace online dedicato agli sviluppatori indipendenti, che consente la distribuzione e l'acquisto di giochi, asset, strumenti e risorse creative (vedi Figura 2.2.4). È particolarmente apprezzato nella comunità indie per la sua facilità d'uso, la varietà di contenuti disponibili, sia gratuiti che non, e la possibilità di supportare direttamente gli artisti. I principali asset grafici sono stati creati dall'artista *penusbmic* [6] permettendo una coerenza estetica tra le varie componenti grafiche del gioco.

I vari font utilizzati sono stati selezionati da Google Fonts [7], un servizio che offre una vasta gamma di caratteri tipografici liberi da utilizzare e con licenze aperte.

¹Per asset si intende tutti gli elementi che compongono un videogioco, come per esempio gli elementi grafici e sonori.

Capitolo 3

Applicazione creata



Figura 3.0.1: Screenshot del gioco

L'applicazione sviluppata è un videogioco progettato per permettere l'esplorazione e la sperimentazione di diversi approcci al salvataggio dello stato di gioco. Sfruttando il Godot Engine, il gioco, intitolato *Hoard*, utilizza meccaniche di gameplay molto semplici che permettono al giocatore di combattere orde di nemici sequenziali.

Nelle seguenti sezioni di questo capitolo verranno descritte le sue principali caratteristiche.

3.1 Meccaniche di gioco

Hoard è basato su delle semplici meccaniche di combattimento in tempo reale che permettono al giocatore di controllare un personaggio, mostrato in Figura 3.0.1. Oltre al movimento, il giocatore è in grado di attaccare, per danneggiare i nemici, e schivare, per evitare i loro attacchi. Il personaggio controllato dal giocatore possiede anche due importanti statistiche mostrate dall'interfaccia grafica: la barra della salute e la barra della stamina. La prima rappresenta i punti vita rimanenti al personaggio, che una volta azzerati fanno perdere la partita. La seconda rappresenta la capacità di effettuare azioni e si rigenera costantemente durante la partita. Ogni volta che il personaggio attacca o schiva, una percentuale di stamina viene sottratta, obbligando il giocatore a gestire accuratamente le proprie azioni.

Attraverso questo sistema di combattimento il giocatore affronterà una serie di ondate di nemici. Una volta sconfitte, otterrà una ricompensa in punti esperienza che gli permetteranno di salire di livello. Il livello del giocatore permette di scalare la difficoltà del gioco in base ai progressi precedenti (più ondate sconfiggerà il giocatore e più la difficoltà aumenterà).

3.2 Nemici

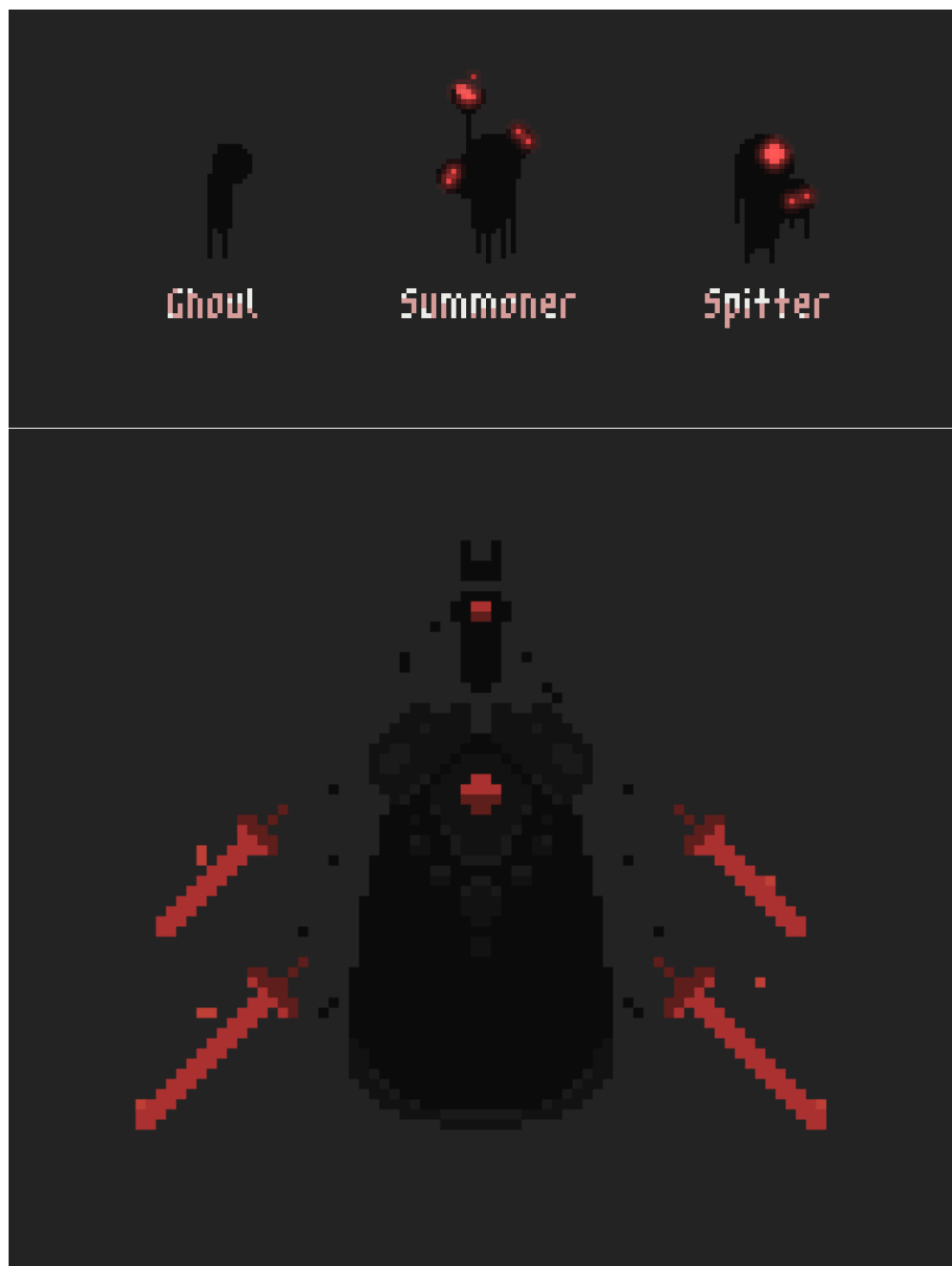


Figura 3.2.2: Tipologie dei nemici, in ordine da in alto a sinistra: Ghoul, Summoner, Spitter e Hearth Hoarder

I nemici in *Hoarde*, proprio come il giocatore, possiedono anch'essi un certo numero di punti vita. Esistono quattro tipologie di nemici all'interno del gioco (vedi Figura 3.2.2), ognuno con i propri attacchi e le proprie caratteristiche:

- **Ghoul**: il nemico base, caratterizzato da un attacco corpo a corpo semplice ma rapido. Ha

una quantità moderata di punti vita ed è veloce negli spostamenti, risultando particolarmente pericoloso in gruppo.

- **Spitter:** un nemico a distanza che sputa proiettili velenosi verso il giocatore. Sebbene più debole fisicamente rispetto al Ghoul, la sua capacità di attaccare da lontano lo rende un avversario insidioso.
- **Summoner:** un nemico che evoca Ghoul e Spitter per attaccare il giocatore. Ha punti vita relativamente alti e può facilmente sopraffare il giocatore se non viene affrontato rapidamente.
- **Hearth Hoarder (Boss):** il boss finale delle ondate. Ha una quantità elevata di punti vita e diversi attacchi ed abilità. Appare solamente nell'ultima ondata della partita.

Capitolo 4

Sistema di salvataggio

Il sistema di salvataggio è un elemento essenziale nei videogiochi, poiché consente di memorizzare in modo persistente i progressi compiuti dal giocatore, permettendo a quest'ultimo di interrompere la propria esperienza di gioco e riprenderla successivamente dal medesimo punto in cui è stata interrotta.

Con l'introduzione degli autosalvataggi, che operano automaticamente dietro le quinte senza richiedere l'interazione diretta del giocatore, il processo è diventato quasi impercettibile e spesso dato per scontato. Tuttavia, il sistema di salvataggio rimane una componente cruciale nell'architettura di un videogioco. La sua implementazione deve non solo garantire l'integrità dei dati salvati, ma anche tenere conto delle proprie prestazioni per assicurare che il salvataggio e caricamento avvengano in modo fluido, per interrompere il meno possibile l'esperienza e l'immersione del giocatore.

Inoltre, il sistema deve essere progettato per supportare la retrocompatibilità, in modo che i giocatori possano mantenere i propri progressi anche in caso di aggiornamenti futuri che potrebbero modificare la struttura o il contenuto dei dati salvati.

Nelle sezioni successive verranno illustrate in dettaglio le caratteristiche, le scelte progettuali e il funzionamento del sistema di salvataggio implementato in *Hoarde*.

4.1 Formati di file

Una scelta importante da fare nelle prime fasi della progettazione di un sistema di salvataggio è sicuramente il formato di file sul quale andare a serializzare i dati. Questa scelta influisce su vari fattori, tra cui l'efficienza delle operazioni di lettura e scrittura, l'occupazione di memoria, la sicurezza dei dati e la facilità con cui i file possono essere modificati.

In questa sezione, verranno esaminati e confrontati i due formati di file più comunemente utilizzati per il salvataggio dello stato di gioco: JSON e il formato binario. Ognuno di questi formati presenta caratteristiche specifiche che ne influenzano la scelta in determinati contesti. Sarà inoltre fornita una panoramica di altri formati potenzialmente utilizzabili sottolineando brevemente le loro principali caratteristiche.

4.1.1 Formato JSON

JavaScript Object Notation (JSON) è un formato di file per lo scambio di dati concepito all'inizio degli anni 2000 per rispondere alla necessità di uno standard che consentisse la comunicazione in tempo reale tra browser e server nel web. Sebbene JSON sia basato su JavaScript, è un formato indipendente e può essere utilizzato con una vasta gamma di linguaggi di programmazione, grazie alle numerose librerie disponibili. [8]

I dati sono memorizzati in una struttura attributo-valore scritta in *plain-text*, rendendolo facilmente leggibile e modificabile dall'uomo.

Il seguente esempio rappresenta i dati di un'ipotetica persona in questo formato [9]:

```
{
  "name": "Mario",
  "surname": "Rossi",
  "active": true,
  "favouriteNumber": 54,
  "birthday": {
    "day": 1,
    "month": 1,
    "year": 2000
  },
  "languages": [ "it", "en" ]
}
```

JSON è ampiamente adottato nelle API moderne e nei sistemi di interscambio dati grazie alla sua semplicità e flessibilità. La natura leggibile e modificabile del formato, combinata con il supporto per strutture complesse come gli array e gli oggetti annidati, lo rende particolarmente adatto per la serializzazione e la trasmissione di dati tra sistemi eterogenei.

4.1.2 File binari

Un file binario è un termine generico utilizzato per indicare tutti quei file che memorizzano dati in un formato non testuale. Solitamente, un file di questo tipo, può essere pensato come una sequenza di byte. Questa modalità di memorizzazione consente operazioni di lettura e scrittura significativamente più rapide rispetto ai file testuali e tende a occupare meno spazio di memoria.

Tuttavia, leggere e interpretare un file binario può risultare più complesso rispetto alla sua controparte testuale, poiché i dati devono essere decodificati secondo uno schema specifico e non sono progettati per essere leggibili direttamente dall'occhio umano. In assenza di uno standard ben definito, la struttura del file, e come essa debba venir interpretata, è determinata liberamente dallo sviluppatore.

I file binari vengono spesso utilizzati per memorizzare una vasta gamma di dati, tra cui immagini, suoni, file compressi e file eseguibili. In generale, questo formato è preferibile in contesti dove l'efficienza e la velocità sono fondamentali, come nel trattamento di dati multimediali, dove la

dimensione ridotta dei file e la rapidità di accesso possono influire significativamente sulle prestazioni complessive dell'applicazione.

4.1.3 Altri formati

Sebbene JSON e binario siano i formati più comuni per i sistemi di salvataggio dei videogiochi, esistono numerose altre opzioni. Tra queste, XML e YAML sono formati testuali che offrono valide alternative a JSON. Il CSV è un altro formato testuale che, sebbene principalmente utilizzato per dati tabulari, può essere impiegato per salvare informazioni semplici in forma strutturata.

Infine, per quanto riguarda l'ambiente di sviluppo del Godot Engine, è opportuno menzionare la possibilità di salvare lo stato di gioco utilizzando le risorse integrate nel motore, note come Godot Resources. Queste offrono un metodo nativo per la serializzazione dei dati, ottimizzato per le esigenze specifiche del motore. Per maggiori dettagli su questa opzione, si rimanda alla documentazione ufficiale di Godot[10].

4.1.4 Confronto tra JSON e binario

La Tabella 1 riassume le principali differenze e considerazioni relative a JSON e file binari descritte in precedenza, evidenziando gli aspetti chiave di entrambi.

Aspetto	JSON	Binario
Leggibilità	Leggibile dall'uomo	Non leggibile dall'uomo
Struttura dei Dati	Gerarchica (oggetti, array)	Definita dallo sviluppatore
Dimensione del File	Maggiore a causa del testo	Più compatto
Prestazioni	Lettura/scrittura più lenta	Lettura/scrittura più veloce
Compatibilità	Più facile da mantenere	Richiede una gestione accurata
Complessità	Meno complesso da usare	Più complesso da implementare

Tabella 4.1: Confronto tra i formati JSON e binario

Per quanto riguarda l'utilizzo di questi file all'interno di Godot, la documentazione ufficiale [11] specifica che JSON supporta solo un insieme limitato di tipi di dati. In caso di dati più complessi, anche nativi del motore grafico (come `Vector2`, `Vector3`, `Color`, `Rect2` e `Quaternion`), sarà necessario tradurli nei tipi supportati. Ad esempio, un `Vector2`, che è la rappresentazione di un vettore bidimensionale all'interno di Godot, potrebbe venir rappresentato come due float separati all'interno di un file JSON.

Il discorso andrebbe a complicarsi in caso di classi molto complesse, come per esempio la classe `Player` all'interno di *Hoarde*, che va a rappresentare l'entità controllata dal giocatore. In questo caso sarebbe richiesta l'implementazione di una logica specifica per tradurre la classe intera, aumentando significativamente la complessità del processo di serializzazione e deserializzazione.

Dato il limitato supporto per la serializzazione in JSON offerto da Godot, anche la documentazione stessa spinge gli sviluppatori ad utilizzare i file binari per progetti di medie-grandi dimensioni.

Tuttavia, JSON può tornare comunque molto utile nella creazione di prototipi o di piccoli progetti, grazie al fatto che i file di salvataggio possono essere letti e modificati in un editor testuale direttamente dallo sviluppatore, facilitando la sperimentazione e il debugging.

4.1.5 Conclusione

In conclusione, si è scelto quindi di utilizzare i file binari. Tuttavia, è importante sottolineare che non esiste una soluzione universalmente corretta.

La decisione su quale formato di file utilizzare dipende strettamente dal contesto specifico di sviluppo, dalle esigenze del progetto e dagli obiettivi che si intendono raggiungere. Ogni soluzione ha i propri vantaggi e svantaggi, che devono essere considerati con attenzione per garantire che la scelta finale sia la più adatta alle esigenze degli sviluppatori.

4.2 Sicurezza

La sicurezza dei file di salvataggio è un tema cruciale e complesso che richiede un'attenta considerazione. Di seguito, vengono brevemente esposti i principali problemi legati alla protezione dei file di salvataggio nei videogiochi, con una distinzione tra videogiochi in locale, ovvero eseguiti sulla macchina dell'utente, e videogiochi online.

Per quanto riguarda i primi, è essenziale riconoscere che nessun sistema installato sulla macchina dell'utente può essere considerato completamente sicuro. Una volta installato un gioco, l'intero software, inclusi eventuali algoritmi di cifratura e chiavi di protezione, diventa potenzialmente accessibile all'utente. Un individuo sufficientemente determinato potrebbe riuscire a estrarre queste informazioni dal codice e accedere ai file di salvataggio.

Pertanto, le strategie di protezione dei file di salvataggio nei giochi in locale sono a discrezione degli sviluppatori. Alcuni optano per non implementare alcuna misura di sicurezza. Altri, invece, preferiscono adottare tecniche di sicurezza per rendere più difficile l'accesso non autorizzato. Nel caso di *Hoarde*, si è scelto di adottare un approccio intermedio.

L'utilizzo di file binari permette di rendere i file di salvataggio non leggibili e difficilmente interpretabili dagli utenti, senza la necessità di ricorrere a tecniche complesse di cifratura. Sebbene, come discusso in precedenza, un utente potrebbe eventualmente riuscire ad accedere ai file, si è deciso di lasciare a quest'ultimo la libertà di farlo, pur consapevoli che ciò potrebbe compromettere la sua stessa esperienza di gioco.

Per quanto riguarda i videogiochi online, il fattore sicurezza è di fondamentale importanza, soprattutto nei contesti multigiocatore, per garantire un'esperienza di gioco equa. In questi casi, è consigliabile memorizzare i file di salvataggio e qualsiasi altro dato sensibile su un server remoto,

al di fuori della portata diretta dell'utente, per garantire una protezione efficace contro accessi non autorizzati.

4.3 Struttura

Il sistema di salvataggio implementato in *Hoarde* adotta una struttura a slot: un approccio comunemente utilizzato che consente al giocatore di gestire fino a tre salvataggi indipendenti tra loro. Ogni slot rappresenta un'istanza separata del gioco, permettendo all'utente di tornare a momenti specifici senza necessariamente sovrascrivere i progressi precedenti.

I file di salvataggio vengono memorizzati localmente sulla macchina dell'utente, utilizzando il percorso specificato dal prefisso `user://` fornito da Godot [12]. Questo prefisso reindirizza a una directory generata automaticamente dall'engine, garantendo che l'applicazione possa sempre scrivere in essa, anche dopo l'esportazione del progetto. Di conseguenza, si assicura un accesso persistente e sicuro ai dati di salvataggio.

Il prefisso `user://` corrisponde a un percorso effettivo nel *file system*, che varia a seconda del sistema operativo in uso. Su piattaforme desktop, i percorsi predefiniti per la cartella `user://` sono i seguenti:

- **Windows:** %APPDATA%\Godot\app_userdata\Hoarde
- **macOS:** ~/Library/Application Support/Godot/app_userdata/Hoarde
- **Linux:** ~/.local/share/godot/app_userdata/Hoarde

Godot offre l'eventuale possibilità di personalizzare e modificare questi percorsi predefiniti all'interno delle impostazioni di progetto. Tuttavia, per lo sviluppo di *Hoarde*, si è deciso di mantenere i percorsi di default forniti dal game engine.

All'interno della cartella specificata da `user://`, si trova una sottocartella denominata `saves`. I file di salvataggio sono contenuti ed organizzati all'interno di essa come mostrato in Figura 4.3.1.

slot_0	18/07/2024 14:08	File folder
slot_1	18/07/2024 14:36	File folder
slot_2	18/07/2024 15:10	File folder

Figura 4.3.1: Struttura della cartella `saves`

save.dat	15/08/2024 15:37	DAT File	3 KB
----------	------------------	----------	------

Figura 4.3.2: File di salvataggio per un generico slot

All'interno della cartella di ciascuno slot può essere presente un file `save.dat` (Figura 4.3.2). Questo file, in formato binario, contiene tutti i dati relativi al salvataggio associato a quello specifico slot.

4.4 Salvataggio

A livello macroscopico, il processo di salvataggio in un videogioco si articola in due fasi principali:

- **Selezione dei dati:** in questa fase è necessario fare una distinzione tra oggetti¹ persistenti e non. Per ognuno degli oggetti persistenti vengono selezionati e raggruppati i dati che occorre salvare.
- **Serializzazione su file:** in questa fase si va effettivamente a serializzare su file i dati raggruppati nella fase precedente.

Nelle sezioni seguenti verranno approfonditi specifici aspetti dell'implementazione del processo di salvataggio di *Hoarde*. Ove necessario, verranno inoltre introdotti alcuni elementi del game engine per chiarire determinati aspetti tecnici che saranno utili anche in seguito, quando si parlerà del processo di caricamento.

4.4.1 SaveManager e autoload

In *Hoarde*, l'intera logica di gestione dei salvataggi è racchiusa all'interno di una classe chiamata **SaveManager**. Poiché questa classe non richiede più istanze durante l'esecuzione del gioco, rappresenta una candidata ideale per l'adozione del design pattern creazionale *singleton*[13].

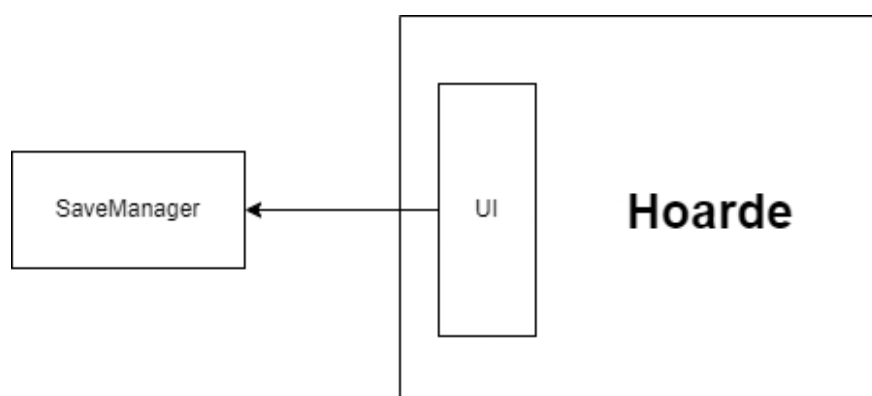


Figura 4.4.3: Rappresentazione grafica dell'interazione tra *Hoarde* e il modulo che gestisce il sistema di salvataggio **SaveManager**

In Godot, il pattern singleton è supportato dal concetto di *autoload*[14]. Un autoload si occupa, infatti, di caricare e mantenere in memoria un'istanza di una scena o uno script, rendendoli accessibili globalmente per tutta l'esecuzione dell'applicazione. Questa scelta risulta perfetta per il nostro sistema di salvataggio perché permette di rendere la classe **SaveManager** un modulo esterno al vero e proprio gioco. Non solo l'accesso alle sue funzionalità sarà possibile da qualsiasi parte del codice, ma potrà anche essere facilmente riadattato ed utilizzato per altri progetti.

¹Da qui in avanti il termine "oggetti" verrà utilizzato come sinonimo degli elementi del gioco, ossia le scene e i nodi di Godot.

4.4.2 Struttura del file

La struttura del file di salvataggio è rappresentata nel codice da una classe chiamata `GameSave`, mostrata qui di seguito:

```
1      class_name GameSave
2
3      # Globali
4      var version: String
5
6      # Mondo di gioco
7      var current_wave: int
8      var time_elapsed: float
9      var killed_enemies: int
10     var to_spawn_enemies: int
11     var is_boss_already_spawned: bool
12
13     # Giocatore
14     var player_health: float
15     var player_stamina: float
16     var player_global_position: Vector2
17
18     # Nemici
19     var enemies: Array[Dictionary]
```

Listing 1: Struttura della classe `GameSave` in GDScript

All'interno della classe sono contenuti tutti i dati necessari per il ripristino dello stato di gioco, suddivisi in quattro macro-categorie:

- **Globali:** contiene informazioni generali relative al gioco.
- **Mondo di gioco:** raccoglie dati sullo stato corrente del mondo di gioco, tra cui il tempo trascorso dall'inizio della partita, la quantità di nemici sconfitti e di quelli ancora in vita.
- **Giocatore:** raccoglie i dati relativi al giocatore, come la salute, la stamina e la posizione all'interno del mondo di gioco.
- **Nemici:** contiene un array di dizionari (strutture dati complesse chiave-valore) in cui ogni elemento di questo array raccoglie i dati di ogni nemico ancora in vita, come la loro salute e la posizione all'interno del mondo.

Attualmente, la classe `GameSave` funge solamente da semplice template. Per poter contenere i dati effettivi, è necessario che la classe venga istanziata, dopodiché a ciascun attributo dell'istanza dovrà essere assegnato un valore specifico. Sebbene istanziare la classe non presenti difficoltà, il processo di popolamento dei dati richiede il recupero di informazioni da vari oggetti distribuiti

all'interno del gioco. Questa procedura, a seconda delle esigenze del progetto, può essere approcciata in diversi modi.

Un primo approccio potrebbe essere inserire un riferimento a ciascuno degli oggetti persistenti all'interno del **SaveManager** e, ogni volta che il giocatore salva, estrapolare i dati da ciascuno di essi, inserirli all'interno di un'istanza di **GameSave** e poi serializzare quest'ultima su file. Tuttavia questo approccio presenta una serie di problemi. Per prima cosa aumenterebbe notevolmente il livello di accoppiamento tra il **SaveManager** e gli altri elementi del gioco, compromettendo l'indipendenza del modulo dall'applicazione discusso in precedenza. Inoltre, si creerebbe un problema di scalabilità: ogni modifica alla struttura dei dati salvati, come l'aggiunta o la rimozione di proprietà, richiederebbe aggiornamenti continui nei riferimenti e nel codice dell'autoload. Senza contare i casi degli elementi dinamici, come i nemici, che una volta sconfitti vengono rimossi, e i loro riferimenti dovrebbero essere eliminati al fine di evitare accessi alla memoria non validi. Questa può essere una valida soluzione per un piccolo progetto che non si evolverà nel tempo, ma risulta poco conveniente per uno che necessita scalabilità.

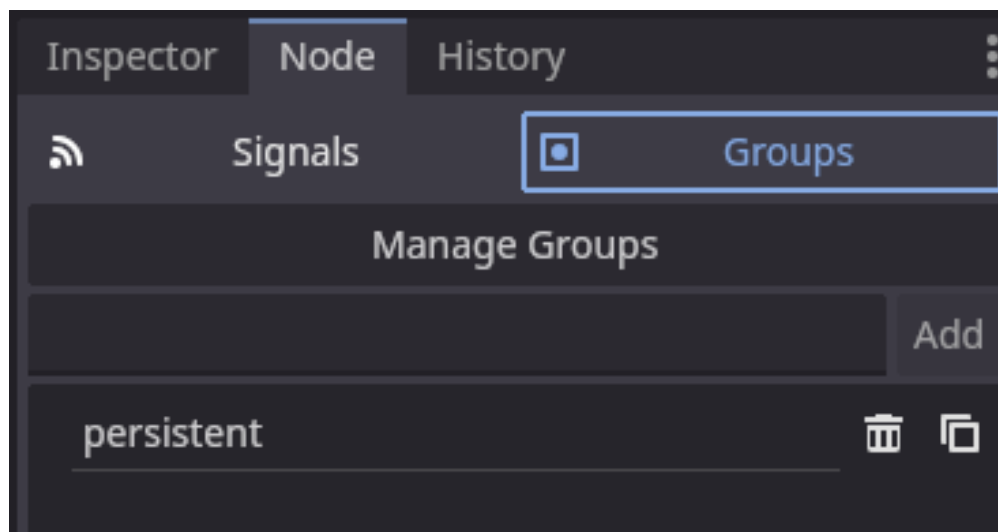
Un secondo approccio potrebbe essere rendere anche la classe **GameSave** un autoload, creando un'unica istanza globalmente accessibile. A questo punto, quando è necessario salvare, ogni oggetto aggiungerà i propri dati all'interno dell'istanza e successivamente il **SaveManager** si occuperà di serializzarla su file. Tuttavia anche questa soluzione, sebbene risolva il problema dell'accoppiamento tra il **SaveManager** e gli oggetti di gioco, presenta altri tipi di problemi. In caso di futuri aggiornamenti del gioco che comporterebbero la crescita del file di salvataggio, mantenere per tutta la durata del gioco in memoria una struttura di elevate dimensioni potrebbe essere dispendioso. Inoltre, si presenterebbe la necessità di gestire la sincronizzazione tra gli oggetti di gioco, il **GameSave** e il **SaveManager**. Per esempio, il **SaveManager** deve serializzare l'istanza di salvataggio solamente dopo che tutti gli oggetti di gioco hanno finito di inserire i loro dati.

Vediamo quindi che trovare una soluzione semplice e scalabile può risultare complicato. Fortunatamente Godot presenta delle funzionalità per gestire in modo semplice ed intuitivo problemi di questo tipo. Nell'engine esiste infatti la possibilità di raggruppare nodi secondo un determinato criterio per poi svolgere operazioni su di essi.

4.4.3 Gruppi

I gruppi in Godot funzionano come etichette assegnabili, consentendo di associare un nodo a un numero illimitato di gruppi (vedi Figura 4.4.4). Organizzare i nodi in gruppi aggiunge nuove opportunità offerte dallo **SceneTree**, come la possibilità di:

- Ottenere una lista di nodi appartenenti a un gruppo
- Chiamare un metodo su tutti i nodi di un gruppo
- Notificare tutti i nodi di un gruppo



```
1 class_name Enemy
2
3 func _ready():
4     add_to_group("persistent")
5     add_to_group("enemies")
```

Figura 4.4.4: L'assegnazione di un nodo a un gruppo in Godot può essere fatta sia dall'editor che mediante codice (righe 4 e 5)

L'utilizzo di queste funzionalità riduce l'accoppiamento e migliora l'organizzazione all'interno dei progetti.

In *Hoarde*, si è scelto di raggruppare tutti gli oggetti di gioco persistenti all'interno di uno stesso gruppo (**persistent**) e sfruttare la possibilità di richiamare lo stesso metodo su ognuno di essi.

Nella prossima sezione verrà illustrato come tutti componenti descritti finora saranno integrati insieme per formare un sistema coeso.

4.4.4 Implementazione

```
1 func save_in_slot(save_slot : SaveSlotNumber):
2     var file = FileAccess.open(
3         get_file_path_from_slot(save_slot),
4         FileAccess.WRITE
5     )
6
7     var game_save = GameSave.new()
8     game_save.version = Global.version
9     get_tree().call_group("persistent", "on_save", game_save)
10    file.store_var(inst_to_dict(game_save))
11
12    file.close()
```

Listing 2: Metodo per il salvataggio dello stato di gioco in GDScript

Il metodo `save_in_slot` è il metodo del `SaveManager` responsabile per il salvataggio dello stato di gioco. Il metodo riceve per parametro dall'interfaccia utente lo slot selezionato dal giocatore nella schemata apposita, mostrata in Figura 4.4.5.

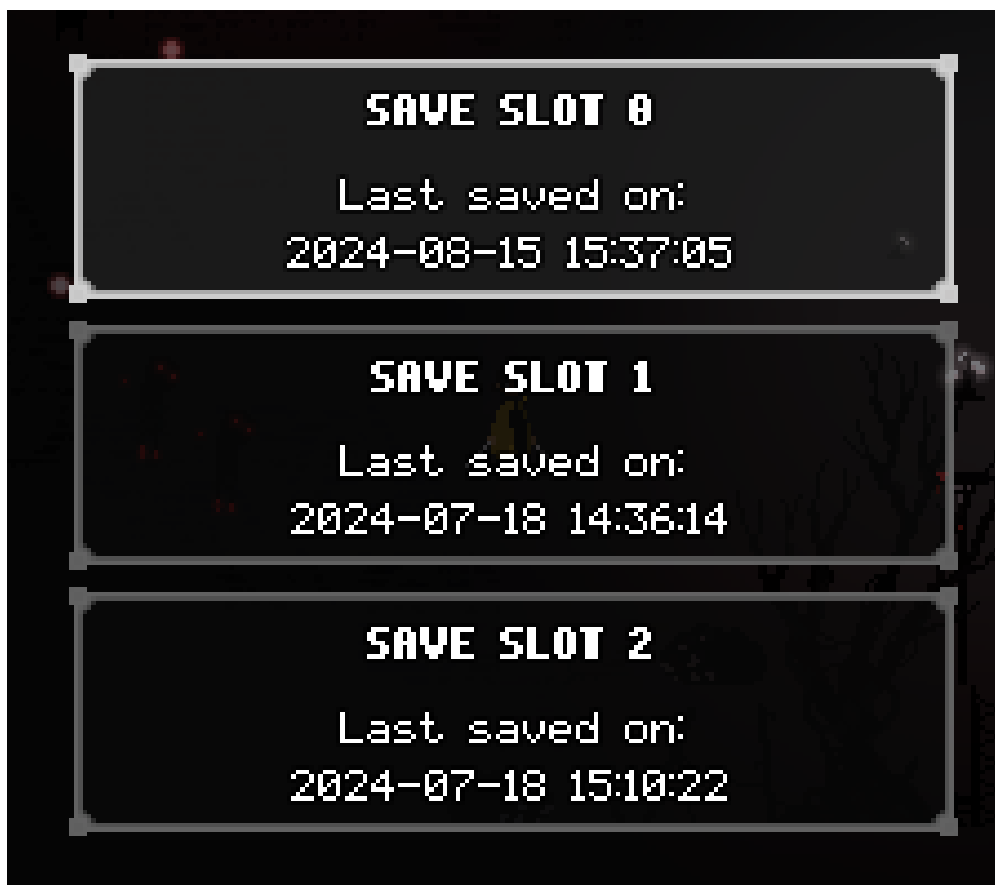


Figura 4.4.5: Schermata dedicata alla selezione dello slot in cui salvare il gioco. Se un salvataggio è già presente in un determinato slot, viene anche mostrata la data e l'ora in cui esso è stato effettuato.

Di seguito sono descritti dettagliatamente i vari passaggi:

1. **Apertura o creazione del file corrispondente allo slot selezionato**

```
var file = FileAccess.open(  
    get_file_path_from_slot(save_slot),  
    FileAccess.WRITE  
)
```

La funzione inizia con l'apertura di un file per la scrittura. Il percorso del file è determinato dal metodo `get_file_path_from_slot(save_slot)`, che restituisce il percorso del file in base al numero di slot di salvataggio fornito. Esso viene mostrato nel Listing 3:

```

1  const SAVE_FOLDER_NAME = "saves"
2  const SAVE_FILE_NAME = "save"
3
4  func get_file_path_from_slot(save_slot : SaveSlotNumber):
5      return "user://" + SAVE_FOLDER_NAME +
6          "/slot_" + str(save_slot) + "/" +
7          SAVE_FILE_NAME + ".dat"

```

Listing 3: Metodo per la conversione dal numero dello slot al percorso del file corrispondente facendo riferimento alla struttura sul *file system* descritta in precedenza.

Se il file non esiste, viene creato automaticamente.

2. Istanziazione di GameSave e popolamento dei dati

```

var game_save = GameSave.new()
game_save.version = Global.version
get_tree().call_group("persistent", "on_save", game_save)

```

Successivamente viene istanziata la classe `GameSave` e popolata coi dati di salvataggio. I dati globali vengono inseriti direttamente dal `SaveManager`, mentre i dati degli oggetti persistenti vengono popolati grazie alle funzionalità offerte dai gruppi.

Dopo aver ottenuto un riferimento allo `SceneTree` grazie al metodo `get_tree`, chiamabile su qualsiasi nodo, il metodo `call_group("persistent", "on_save", game_save)` invocherà, se implementato, il metodo `on_save` su tutti gli oggetti appartenenti al gruppo "persistent". L'istanza dell'oggetto `game_save` viene passata come parametro del metodo, consentendo a ciascun oggetto di aggiungere i propri dati.

```

1  class_name Player
2
3  func on_save(game_save : GameSave):
4      game_save.player_health = current_health
5      game_save.player_stamina = current_stamina
6      game_save.player_global_position = global_position

```

```

1  class_name Enemy
2
3  func on_save(game_save : GameSave):
4      var dict = {
5          "type": get_type(),

```

```
6         "is_summoned": is_summoned,  
7         "global_position": global_position,  
8         "health": health  
9     }  
10    game_save.enemies.append(dict)
```

Listing 4: Implementazione del metodo `on_save` sulle classi `Player` ed `Enemy`.

3. Serializzazione su file

```
file.store_var(inst_to_dict(game_save))
```

Dopo aver raccolto tutti i dati necessari, l'istanza `game_save` viene convertita in un dizionario² tramite la funzione `inst_to_dict`. Questo dizionario viene quindi serializzato nel file binario utilizzando `store_var`.

4. Chiusura del file:

```
file.close()
```

Infine, il file viene chiuso per garantire che tutte le modifiche siano salvate e che le risorse siano rilasciate correttamente.

Alla fine di questo processo ci si ritrova con il file binario contenente i dati di salvataggio presente sul disco, permettendo al giocatore di ripristinarli in qualsiasi momento tramite il processo di caricamento, approfondito nella prossima sezione.

4.5 Caricamento

Il processo di caricamento è, per sua natura, l'inverso di quello di salvataggio e può essere anch'esso suddiviso in due fasi distinte:

- **Deserializzazione da file:** in questa fase vengono letti i dati contenuti nel file di salvataggio associato allo slot selezionato dal giocatore.
- **Ripristino dello stato di gioco:** in questa fase si utilizzano i dati letti nella precedente fase per ricreare lo scenario salvato.

Sebbene le strutture utilizzate e le operazioni eseguite siano pressoché simili a quelle descritte in precedenza, esistono alcuni aspetti specifici del caricamento che meritano un'attenzione particolare.

²A causa di problemi presenti in Godot 4.2.1 con la serializzazione diretta di istanze di oggetti annidati su file binario, si è optato per questa conversione intermedia in dizionario. Questa scelta, sebbene inizialmente forzata, semplificherà la soluzione dei problemi discussi nella sezione sulla retrocompatibilità.

Nella seguente sottosezione, come per il processo di salvataggio, verranno descritti dettagliatamente i vari passaggi che compongono l'implementazione del processo di caricamento in *Hoarder*, con particolare enfasi sulla parte più complessa, ovvero il cambio di scene.

4.5.1 Implementazione

```
1 func load_from_slot(save_slot : SaveSlotNumber):  
2     var file = FileAccess.open(  
3         get_file_path_from_slot(save_slot),  
4         FileAccess.READ  
5     )  
6     if file == null:  
7         print("Error: no save file to load from")  
8         return  
9  
10    var game_save = dict_to_inst(file.get_var()) as GameSave  
11    change_to_loaded_scene(game_save)  
12    file.close()
```

Listing 5: Metodo per il ripristino dello stato di gioco in GDScript

Il metodo `load_from_slot`, contenuto all'interno del `SaveManager`, contiene tutta la logica riguardante il caricamento. Proprio come il metodo `save_in_slot`, responsabile del salvataggio, anch'esso riceve come parametro il numero dello slot selezionato dal giocatore in una schermata identica a quella mostrata precedentemente in Figura 4.4.5.

Di seguito sono descritti dettagliatamente i vari passaggi:

1. Apertura del file:

```
var file = FileAccess.open(  
    get_file_path_from_slot(save_slot),  
    FileAccess.READ  
)  
if file == null:  
    print("Error: no save file to load from")  
    return
```

Anche in questo caso, il percorso del file viene determinato, in base allo slot scelto, tramite il solito metodo `get_file_path_from_slot`, riportato in precedenza. Se il file esiste, viene aperto in lettura, altrimenti viene restituito un errore e l'operazione viene interrotta.

2. Deserializzazione del file:

```
var game_save = dict_to_inst(file.get_var()) as GameSave
```

In caso di apertura con successo del file, il contenuto viene deserializzato con un processo inverso a quello adottato durante il salvataggio. Con il metodo `dict_to_inst` il dizionario ottenuto dalla deserializzazione del file viene convertito in un'istanza della classe `GameSave`, con i valori dei rispettivi attributi già assegnati, ricostruendo con successo l'istanza salvata in precedenza.

3. Cambio di scena e ripristino dello stato di gioco:

```
change_to_loaded_scene(game_save)
```

Una volta riottenuta l'istanza, per ricostruire lo scenario salvato in precedenza sono necessari tre³ passaggi:

- Creazione di una nuova istanza "pulita" della scena che rappresenta il mondo di gioco (Figura 4.5.6).
- Sostituzione della scena corrente (quella in cui il giocatore si trova quando decide di caricare un salvataggio) con la scena appena istanziata.
- Ripristino dello stato di gioco nella nuova scena corrente in base ai dati letti dal file di salvataggio, e ora presenti nell'istanza di `GameSave`.

³In base a come sono state progettate le singole componenti, le interazioni tra di esse e la complessità generale del gioco, in alcuni casi sarebbe possibile andare direttamente a ripristinare lo stato di gioco sulla scena corrente (saltando quindi sia il primo che il secondo passaggio). Nel nostro caso, è stato necessario utilizzare tutti e tre passaggi. Si noti che in entrambi i casi si tratta di sole differenze implementative che, se implementate correttamente, non andrebbero a cambiare la percezione del giocatore.

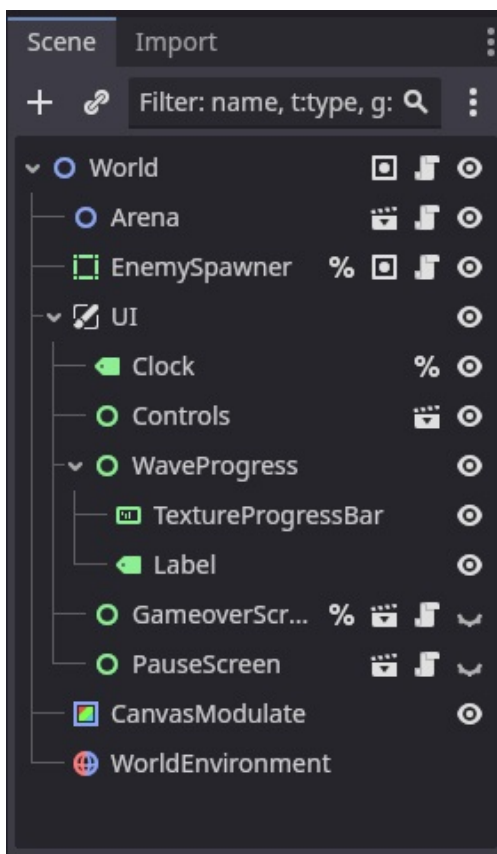


Figura 4.5.6: Scena che rappresenta il mondo di gioco (esclusi i nodi aggiunti a runtime come il giocatore e i nemici) vista dall'editor di Godot

Il metodo `change_to_loaded_scene` si occupa proprio di questi tre passaggi:

```

1  const WORLD_SCENE = preload("res://world.tscn")
2
3  func change_to_loaded_scene(game_save : GameSave):
4      # Primo passaggio
5      var tree = get_tree()
6      var current_scene = tree.current_scene
7      var new_scene = WORLD_SCENE.instantiate()
8      if tree.paused:
9          tree.paused = false
10
11     # Secondo passaggio
12     if current_scene != null:
13         current_scene.queue_free()
14     new_scene.is_loaded = true
15     tree.root.add_child(new_scene)
16     tree.current_scene = new_scene
17
18     # Terzo passaggio

```

```
19 tree.call_group("persistent", "on_load", game_save)
```

Listing 6: Metodo per il ripristino dello stato di gioco e il cambio di scena

Nel primo passaggio (righe 6-10), vengono preparate le due scene coinvolte. La nuova istanza della scena del mondo di gioco viene facilmente creata a partire dal suo file risorsa⁴. Un riferimento alla scena corrente è invece ottenibile grazie alla proprietà `current_scene` dello `SceneTree`, accessibile tramite `get_tree` da qualsiasi nodo. Infine, tenendo in considerazione che le schermate di salvataggio e caricamento sono spesso utilizzate dal menù di pausa, il gioco viene riattivato se necessario (`tree.paused = false`).

Il secondo passaggio (righe 13-17) prevede il vero e proprio cambio di scena. La scena corrente viene quindi rimossa dallo `SceneTree` e dalla memoria con il metodo `queue_free`, mentre la nuova scena viene aggiunta e impostata come corrente.

Nel terzo passaggio (riga 20) viene ripristinato lo stato di gioco all'interno della nuova scena seguendo lo stesso principio basato sui gruppi, già applicato in fase di salvataggio. Il metodo `call_group` viene quindi usato per invocare il metodo `on_load` sugli oggetti persistenti affinché eseguano il ripristino del loro stato in base ai dati contenuti nell'istanza di `GameSave`.

```
1 class_name EnemySpawner
2
3 func reset_enemy(enemy : Dictionary):
4     var new_enemy: Enemy = string_to_instance(enemy["type"])
5     new_enemy.global_position = enemy["global_position"]
6     new_enemy.health = enemy["health"]
7     var world = get_parent()
8     if not enemy["is_summoned"]:
9         new_enemy.died.connect(world._on_enemy_died)
10    world.add_child(new_enemy)
11
12 func on_load(game_save : GameSave):
13    print("Loading enemies...")
14    for enemy in game_save.enemies:
15        reset_enemy(enemy)
16    print("Enemies loaded!")
```

Listing 7: Implementazione del metodo `on_load` sulla classe `EnemySpawner`, responsabile della gestione dei nemici.

4. Chiusura del file:

⁴Si ricorda che le scene di Godot sono salvate sottoforma di file risorsa (riconoscibili dall'estensione `.tscn`), permettendo di pre-caricarle all'interno di variabili, come mostrato a riga 1, al fine di istanziarle a runtime.

```
file.close()
```

Infine, il file viene chiuso e le risorse rilasciate.

4.6 Retrocompatibilità

Nella maggior parte dei casi, il ciclo di sviluppo di un videogioco non si conclude con il suo rilascio sul mercato. Anzi, una volta pubblicato, gli sviluppatori continuano a mantenerlo e aggiornarlo per periodi di tempo variabili a seconda del progetto. Gli aggiornamenti possono avere scopi diversi: dalla correzione di bug riscontrati dopo il lancio all'aggiunta di nuovi contenuti. Con il susseguirsi di questi aggiornamenti, è frequente che alcune componenti del gioco subiscano modifiche, vengano introdotti elementi nuovi o altre parti vengano rimosse. Questo processo influisce direttamente sul sistema di salvataggio del gioco. Ogni modifica alle componenti di gioco, introdotta da un aggiornamento, richiede agli sviluppatori di garantire che i salvataggi creati con le versioni precedenti rimangano compatibili con la nuova versione. La complessità di questo processo dipende da diversi fattori, come l'architettura del sistema di salvataggio e la natura degli aggiornamenti stessi. Non è raro, purtroppo, che alcuni giochi, dopo un aggiornamento, corrompano i file di salvataggio, costringendo i giocatori a ricominciare da capo.

Nel nostro caso, sebbene *Hoarde* non sia un gioco particolarmente grande e complesso, abbiamo cercato di simulare diverse tipologie di aggiornamenti sulla versione base del gioco per poter discutere possibili soluzioni. Tali discussioni saranno naturalmente incentrate sul sistema di salvataggio presentato nelle sezioni precedenti di questo capitolo, ma potrebbero comunque fornire indicazioni utili anche per affrontare le stesse problematiche in un contesto più generale. Le tipologie di aggiornamento che analizzeremo in questa sezione sono quattro, ciascuna con un diverso impatto sul file di salvataggio, ovvero:

- Aggiunta di proprietà
- Rimozione di proprietà
- Rinomina di proprietà
- Modifica del dominio di proprietà

4.6.1 Aggiunta di proprietà

In questo primo caso, simuleremo un aggiornamento che introduce nuovi contenuti e, di conseguenza, nuove proprietà nel file di salvataggio.

Si ricorda che il loop di gioco⁵ di *Hoarde* è il seguente: il giocatore affronta quattro ondate di nemici successive e una volta sconfitte tutte vince la partita. Nonostante le ondate di nemici presentino sempre un fattore di casualità, se il gioco rimanesse così risulterebbe ripetitivo dopo

⁵La sequenza di azioni che il giocatore ripete per tutta la durata del gioco.

un paio di partite. Per ovviare a questo problema, si decide di introdurre un nuovo elemento: l'esperienza.

Adesso ogni volta che il giocatore vince una partita, ottiene un certo numero di punti esperienza. Maggiore è l'esperienza accumulata, maggiore sarà la difficoltà del gioco. Sebbene questo meccanismo sia semplice, contribuirebbe ad aumentare il livello di sfida, rendendo il gioco più coinvolgente.

Questo nuovo elemento verrà quindi aggiunto sottoforma di variabile all'interno del gioco e sarà incluso nel sistema di salvataggio (altrimenti ogni volta che il giocatore caricherebbe la partita ripartirebbe sempre da zero). Non si andrà nei dettagli per quanto riguarda la parte di implementazione nel gioco stesso, ma l'aggiunta nel sistema di salvataggio avviene includendo le seguenti linee di codice nei metodi e nelle classi già viste in precedenza:

```
1  class_name GameSave
2
3  # Global
4  var version: String
5  var exp: float
6
7  # World
8  var current_wave: int
9  var time_elapsed: float
10 var killed_enemies: int
11 var to_spawn_enemies: int
12 var is_boss_already_spawned: bool
13
14 # Player
15 var player_health: float
16 var player_stamina: float
17 var player_global_position: Vector2
18
19 # Enemies
20 var enemies: Array[Dictionary]
```

```
1  func save_in_slot(save_slot : SaveSlotNumber):
2  ...
3  game_save.exp = Global.exp
4  ...
```

```
1  func load_from_slot(save_slot : SaveSlotNumber):
2  ...
3  var game_save = dict_to_inst(file.get_var()) as GameSave
4  Global.exp = game_save.exp
5  ...
```

Listing 8: Aggiunta dell'esperienza nel sistema di salvataggio

Avendo progettato il sistema di salvataggio in modo facilmente scalabile queste sono le uniche modifiche necessarie per adattarlo alla nuova versione di gioco.

La situazione che si vuole analizzare è la seguente: supponiamo di caricare un vecchio salvataggio, privo del campo appena aggiunto, nella nuova versione di gioco. Il problema che può sorgere riguarda la lettura di un campo non presente nel file di salvataggio. Questo non è un problema esclusivo dei sistemi di salvataggio nei videogiochi e, a seconda del formato del file e delle tecniche di lettura adottate, esistono diverse soluzioni.

Nel nostro caso, il metodo `dict_to_inst` di Godot permette di risolvere questo problema. Esso infatti, prima crea un'istanza dell'oggetto `GameSave`, poi assegna i valori del dizionario ai rispettivi attributi della classe. Alla fine del processo, i campi mancanti nel file di salvataggio risulteranno comunque inizializzati nell'istanza con i loro valori predefiniti. In questo caso zero, poiché il giocatore non ha mai accumulato esperienza.

```
1 static inline void dict_to_inst(Variant *r_ret, const Variant **p_args,  
2 int p_arg_count, Callable::CallError &r_error) {  
3  
4     ...  
5  
6     for (KeyValue<StringName, GDScript::MemberInfo> &E  
7         : gd_ref->member_indices)  
8     {  
9         if (d.has(E.key)) {  
10             ins->members.write[E.value.index] = d[E.key];  
11         }  
12     }  
13 }
```

Listing 9: Implementazione del metodo `dict_to_inst` interna a Godot (scritto in C++) [15]. La parte finale del metodo itera su ogni attributo della classe (righe 6-11) andando ad assegnargli il corrispettivo valore contenuto nel dizionario (riga 10). Si noti che l'assegnazione occorre solo se esiste una corrispondenza tra il campo della classe e il campo del dizionario (riga 9).

Possiamo quindi concludere che, senza necessità di ulteriori modifiche al nostro sistema di salvataggio, è possibile aggiungere nuovi campi al file senza problemi, prestando comunque la dovuta attenzione ai loro valori di inizializzazione.

4.6.2 Rimozione di proprietà

Supponiamo ora di voler eliminare una proprietà già presente nel file di salvataggio, come per esempio il campo che rappresenta la stamina del giocatore. Oltre all'ovvia rimozione dalla classe `GameSave`, è fondamentale ricordarsi di rimuovere la scrittura e la lettura del campo anche dai metodi `on_save` e `on_load` della classe `Player`.

Se ora tentiamo di caricare un vecchio salvataggio, ci troviamo di fronte a una situazione simile a quella precedente. Se si aggiunge una proprietà, infatti, ci si ritrova con un campo in più nella classe `GameSave` rispetto al file di salvataggio, mentre ora, ci si ritrova nella situazione opposta. Tuttavia, anche in questo caso, il problema viene risolto dal metodo `dict_to_inst`. Durante la conversione da dizionario a istanza, infatti, vengono iterati e inizializzati solamente i campi della classe `GameSave`, ignorando eventuali campi aggiuntivi presenti nel dizionario.

4.6.3 Rinomina di proprietà

Fino a questo punto, sempre tenendo in considerazione la nostra implementazione, i problemi causati dall'aggiunta e dalla rimozione di campi nel file di salvataggio si sono rivelati facilmente risolvibili grazie al metodo `dict_to_inst`. Tuttavia, in questa sottosezione, così come nella successiva, saranno presentati problemi che richiedono soluzioni più elaborate.

Supponiamo che, per motivi di organizzazione interna al progetto, si decida di rinominare un campo. Consideriamo, ad esempio, quello che rappresenta l'esperienza accumulata dal giocatore, attualmente denominato `exp`. Si decide di rinominarlo in `player_experience` per migliorarne la descrittività⁶. Se ripetiamo il consueto procedimento, scopriremo che, durante la conversione da dizionario a istanza, il campo `exp`, non trovando una corrispondenza con un campo omonimo nella classe `GameSave`, verrà completamente ignorato. Ci ritroveremo, pertanto, con l'attributo `player_experience` inizializzato a zero, causando una perdita di informazioni che invece erano state precedentemente salvate.

Una possibile soluzione per affrontare questo problema prevede la solita lettura del dizionario dal file binario ma, prima di procedere alla conversione, viene analizzato il suo contenuto (ad esempio, il numero di versione) ed eventualmente attuata una migrazione dalla vecchia alla nuova versione. Nel caso che stiamo considerando, la migrazione corrisponderebbe alla rinomina del vecchio campo `exp` in `player_experience`, in modo che venga riconosciuta la corrispondenza durante la successiva conversione. Questo procedimento richiede che gli sviluppatori tengano traccia dei cambiamenti della struttura del file con il susseguirsi degli aggiornamenti e, se necessario, implementino un sistema di migrazione da versione a versione. In questo modo, si garantirebbe che qualsiasi file precedente alla versione attuale possa essere aggiornato in modo sequenziale (ad esempio, dalla versione 1.0 alla 1.1 e poi dalla 1.1 alla 2.0) prima di essere letto.

```
1 func load_from_slot(save_slot : SaveSlotNumber):  
2 ...  
3 var dictionary = file.get_var()  
4 dictionary = migrate(dictionary)  
5 var game_save = dict_to_inst(dictionary) as GameSave  
6 Global.exp = game_save.exp  
7 ...  
8
```

⁶Nella maggior parte dei casi, i campi del file di salvataggio vengono rinominati per motivi organizzativi o a vantaggio degli sviluppatori. Il giocatore, infatti, non ha nessuna necessità di conoscere la struttura del file.

```
9 func migrate(dict: Dictionary) -> Dictionary:
10     var version = dict.get("version")
11     match version:
12         "1.0": dict = migrate_from_10_to_11(dict)
13     return dict
14
15 func migrate_from_10_to_11(dict: Dictionary) -> Dictionary:
16     dict.version = "1.1"
17     if dict.has("exp"):
18         dict["player_experience"] = dict["exp"]
19         dict.erase("exp")
20     return dict
```

Listing 10: Una possibile soluzione per il problema, implementata all'interno del nostro sistema di salvataggio

```
1 func load_from_slot(save_slot : SaveSlotNumber):
2     ...
3     var dictionary = file.get_var()
4     dictionary = migrate(dictionary)
5     var game_save = dict_to_inst(dictionary) as GameSave
6     Global.exp = game_save.exp
7     ...
8
9 func migrate(dict: Dictionary) -> Dictionary:
10     var version = dict.get("version")
11     match version:
12         "1.0":
13             dict = migrate_from_10_to_11(dict)
14             dict = migrate_from_11_to_20(dict)
15         "1.1":
16             dict = migrate_from_11_to_20(dict)
17     return dict
```

Listing 11: Esempio di migrazione sequenziale dalla versione 1.0 alla versione 2.0

4.6.4 Cambio di dominio di proprietà

L'ultimo problema che andremo a trattare in questa sezione riguarda il cambio di dominio di un campo del file di salvataggio. Per esempio, attualmente la posizione del giocatore è memorizzata in un vettore bidimensionale `Vector2`, supponiamo che, per qualsiasi motivo, venga cambiato il suo tipo in un vettore tridimensionale `Vector3`.

È chiaro che, anche in questo caso, qualora venisse caricato un vecchio salvataggio, il metodo `dict_to_inst` non sarebbe in grado di trovare una corrispondenza tra il `Vector2` del file e il `Vector3`

della classe `GameSave`. Questo risulterebbe in un'assegnazione non valida e, di conseguenza, una perdita di dati.

Per affrontare questa situazione, si può applicare una strategia simile a quella già discussa in precedenza riguardo alla migrazione dei campi. Prima della conversione del dizionario in istanza, si dovrà esaminare il contenuto del dizionario per verificare la presenza del campo `player_position`. Se esiste, sarà necessario effettuare una conversione manuale da `Vector2` a `Vector3`. In questo caso, ne risulta una banale conversione in cui i valori delle coordinate `x` e `y` verranno preservati, mentre la coordinata `z` potrà essere inizializzata a zero, poiché non facente parte della struttura originale. In contesti di sviluppo più complessi, è possibile che queste conversioni coinvolgano intere classi, aumentando così la complessità del processo. Tuttavia, il principio di una migrazione graduale da versione a versione rimane una soluzione fondamentale, alla base anche delle situazioni più elaborate.

```
1 func load_from_slot(save_slot : SaveSlotNumber):
2 ...
3 var dictionary = file.get_var()
4 dictionary = migrate(dictionary)
5 var game_save = dict_to_inst(dictionary) as GameSave
6 Global.exp = game_save.exp
7 ...
8
9 func migrate(dict: Dictionary) -> Dictionary:
10     var version = dict.get("version")
11     match version:
12         "1.0": dict = migrate_from_10_to_11(dict)
13     return dict
14
15 func migrate_from_10_to_11(dict: Dictionary) -> Dictionary:
16     dict.version = "1.1"
17     if dict.has("player_position"):
18         var old_position = dict["player_position"] as Vector2
19         dict["player_position"] = Vector3(
20             old_position.x,
21             old_position.y,
22             0)
23     return dict
```

Listing 12: Una possibile soluzione per il problema, implementata all'interno del nostro sistema di salvataggio

Capitolo 5

Conclusione

Il lavoro svolto in questa tesi ha portato alla progettazione e implementazione di un sistema di salvataggio scalabile e retrocompatibile per il videogioco *Hoarde*, sviluppato con il game engine Godot. Il sistema di salvataggio, basato sulla serializzazione dei dati e sulla gestione automatica di eventuali modifiche o aggiunte future, ha mostrato di essere flessibile. Questo sistema garantisce che i progressi del giocatore vengano salvati e caricati correttamente, anche in presenza di aggiornamenti significativi al gioco, soddisfacendo così le esigenze iniziali.

Nonostante il buon funzionamento del sistema di salvataggio, alcune difficoltà sono emerse durante lo sviluppo. Come già menzionato in precedenza, sono sorti problemi legati ai metodi per la serializzazione di oggetti annidati presenti nell'interfaccia per la serializzazione binaria di Godot. La soluzione adottata (la conversione intermedia da istanza di oggetto a dizionario) oltre a risolvere il problema precedente, ha anche indirettamente facilitato la risoluzione dei problemi causati dalla retrocompatibilità. Se avessimo serializzato direttamente gli oggetti annidati, infatti, i controlli sui singoli campi avrebbero sicuramente richiesto soluzioni più complesse.

Per quanto riguarda lo sviluppo del gioco, la maggior parte dei problemi emersi coinvolgono aspetti tecnici di Godot che, essendo un engine relativamente giovane, a volte può presentare delle mancanze, magari già colmate in altri engine più anziani, che devono essere aggirate o compensate manualmente dagli sviluppatori.

In prospettiva futura, il sistema di salvataggio potrebbe essere ulteriormente modificato e migliorato. Una delle possibili estensioni consisterebbe nell'implementazione di una funzione di autosalvataggio alla fine di ogni ondata di nemici. Questa caratteristica non solo aumenterebbe la difficoltà, costringendo il giocatore a sconfiggere l'ondata senza possibilità di riavviare da un punto intermedio in caso di sconfitta, ma migliorerebbe anche l'esperienza di gioco, riducendo la necessità di intervento manuale. Inoltre, un'ulteriore evoluzione del sistema potrebbe includere la possibilità di salvare i progressi su una piattaforma cloud, permettendo al giocatore di accedere ai propri dati da dispositivi diversi, incrementando sia la sicurezza che la portabilità dei file di salvataggio.

In conclusione, pur essendo un gioco semplice, lo sviluppo di *Hoarde* e del suo sistema di salvataggio sono stati un'ottima esperienza formativa che ha permesso di imparare i ragionamenti

presenti dietro alla progettazione e alla creazione di tutte le singole parti che compongono un videogioco.

Bibliografia

- [1] Godot Engine. *Godot Engine - Free and open source 2D and 3D game engine*. <https://godotengine.org>.
- [2] Godot Engine. *Godot Docs*. <https://docs.godotengine.org/en/stable/index.html>. Documentazione ufficiale di Godot.
- [3] Godot Engine. *Godot Docs - Nodes and Scenes*. <https://docs.godotengine.org/en/stable/index.html>. Pagina della documentazione ufficiale sui nodi e le scene.
- [4] Godot Engine. *Godot Docs - Using Scene Tree*. https://docs.godotengine.org/en/stable/tutorials/scripting/scene_tree.html. Pagina della documentazione ufficiale sul SceneTree.
- [5] itch.io. *itch.io - Indie game hosting marketplace*. <https://itch.io>.
- [6] penusbmic. *penusbmic itch.io page*. <https://penusbmic.itch.io/>.
- [7] Google Fonts. *Google Fonts*. <https://fonts.google.com>.
- [8] JSON.org. *JSON: The JavaScript Object Notation*. <https://www.json.org>. Il sito include una descrizione dettagliata delle specifiche del formato JSON e fornisce un elenco di librerie per la scrittura e il parsing in vari linguaggi di programmazione.
- [9] Wikipedia. *JavaScript Object Notation*. <https://en.wikipedia.org/wiki/JSON>.
- [10] Godot Engine. *Godot Docs - Resources*. <https://docs.godotengine.org/en/stable/tutorials/scripting/resources.html>. Pagina della documentazione ufficiale sul funzionamento e la gestione delle risorse in Godot.
- [11] Godot Engine. *Godot Docs - Saving games*. <https://docs.godotengine.org/en/stable/tutorials/scripting/resources.html>. Pagina della documentazione ufficiale sul salvataggio dello stato di gioco.
- [12] Godot Engine. *Godot Docs - Data Paths*. Pagina della documentazione ufficiale sui data path di Godot. URL: https://docs.godotengine.org/en/stable/tutorials/io/data_paths.html.
- [13] Robert Nystrom. *Game Programming Patterns*. URL: <https://gameprogrammingpatterns.com/singleton.html>.

-
- [14] Godot Engine. *Godot Docs - Singletons(Autoload)*. Pagina della documentazione ufficiale sugli autoload. URL: https://docs.godotengine.org/en/stable/tutorials/scripting/singletons_autoload.html.
- [15] Godot Engine. *Github - Godot Engine Source Code Repository*. Repository ufficiale del codice sorgente di Godot. In particolare viene citata l'implementazione del metodo `dict_to_inst` (righe 314-388) del file `modules/gdscript/gdscript_utilityfunctions.cpp`. URL: <https://github.com/godotengine/godot>.